

FULL-STACK DEVELOPMENT PLAN

Kids Bible Quiz

A mobile quiz application for children aged 5–12, with a PostgreSQL-backed question bank and a web admin dashboard.

DOCUMENT VERSION	1.3
STATUS	Living document — updated as development progresses
SOURCE SPECIFICATION	Kids_Bible_Quiz_Claude_Build_Spec.md
REPOSITORY	github.com/atebelucky123-cmd/kids-bible-quiz
PREPARED WITH	Claude Code
DATE	16 September 2026

This plan tracks the current implementation approach for the project described in the client's handwritten notes and the derived build specification. Sections marked “developer decision” are provisional and open to revision.

Table of Contents

1. Project Overview	3
2. System Architecture Overview	4
3. Technology Stack	5
4. Development Phases — Summary	6
5. Detailed Phase Descriptions	7
Phase 1 — Project Foundation & Environment Setup	8
Phase 2 — Database Architecture & PostgreSQL Schema	9
Phase 3 — Backend API Foundation	10
Phase 4 — Authentication & User Registration	11
Phase 5 — Mobile Application Foundation & Navigation	12
Phase 6 — Student Home & Session Management	13
Phase 7 — Quiz Engine — Question Delivery & Timing	14
Phase 8 — Answering, Feedback & Progress Persistence	15
Phase 9 — Scoring & Results	16
Phase 10 — Audio, SVG & Visual Feedback Integration	17
Phase 11 — Admin Dashboard	18
Phase 12 — Security Hardening & Validation	19
Phase 13 — Integration & End-to-End Testing	20
Phase 14 — Production Deployment & Handover	21
6. Dependencies Between Phases	22
7. Testing Checkpoints	23
8. Final Definition of Done	24
9. Deployment & Handover Considerations	25
Appendix A — Open Requirements	26
Appendix B — Asset Request List	27
Appendix C — UI Design Reference	29

1. Project Overview

Kids Bible Quiz is a mobile application that lets children aged 5–12 register, take timed multiple-choice Bible quizzes drawn from a PostgreSQL database, and receive immediate, child-friendly feedback. A companion web admin dashboard lets the project owner manage the question bank, view registered students, and review quiz results without editing code or touching the database directly.

This plan is derived from the client's original handwritten notes and the resulting technical build specification. Where a requirement is explicitly stated by the client, this document preserves it exactly. Where a detail was left open (an exact timer duration, exact age bands, hosting choice, and similar), this document records the provisional developer decision separately and flags it so it is never mistaken for a client instruction.

EXPLICIT CLIENT REQUIREMENTS

- iOS-compatible mobile application with a database storage system.
- Registration collecting first/middle/last name, age, mobile number, hobbies, favourite colour, favourite animal, and an alphanumeric password.
- Login using first name and password.
- Age restricted to 5–12; ages above 12 rejected with “Oops, the age is too high” and a smiley face.
- Quiz questions with grading, drawn from a populatable database — up to 70 questions.
- A configurable time limit for questions.
- Correct answers show “Well done” with a star; wrong answers show “Whoops! That is the wrong answer, let's try again.”
- Persistent quiz status: finished / continue.
- A cheers/claps sound when the final grade is above 70%.

2. System Architecture Overview

The system follows a conventional three-tier architecture: a React Native mobile client, a Node.js/Express REST API, and a PostgreSQL database accessed through Prisma. A separate React web admin dashboard talks to the same API through admin-scoped, authorization-protected endpoints. Firebase and Supabase are explicitly excluded from this architecture, per the client's build specification.

```
React Native / Expo mobile app
  |
  | HTTPS / JSON
  v
Node.js + Express REST API
  |
  | Prisma ORM
  v
PostgreSQL database
  ^
  |
React web Admin Dashboard
```

The mobile app never determines the authoritative quiz score and never receives the correct answer for a question ahead of grading — both are enforced server-side. The admin dashboard is a separate application from the student-facing mobile app, sharing only the backend API and database.

3. Technology Stack

All items below are developer decisions made to satisfy the client's architectural constraints (no Firebase/Supabase, a conventional backend and PostgreSQL database), not requirements dictated verbatim by the client.

Layer	Technology	Notes
Mobile app	React Native + Expo (TypeScript)	iOS-compatible per the client's stated project type.
Mobile navigation	Expo Router	File-based routing matching the screen flow.
Backend API	Node.js + Express (TypeScript)	Conventional REST API, no GraphQL.
Database	PostgreSQL	Firebase/Supabase explicitly excluded.
ORM	Prisma	Schema, migrations, and typed queries.
Validation	Zod	Shared validation conventions across all endpoints.
Authentication	Backend-managed sessions	Secure password hashing; HTTP-only session cookies where practical.
Admin dashboard	React + TypeScript	Separate web application, same backend API.
DB administration	PostgreSQL / pgAdmin	Database stays directly manageable outside the app, including in production via the Neon connection string.
API testing	Postman	Manual and regression testing of all endpoints.
Version control	Git / GitHub	github.com/atebelucky123-cmd/kids-bible-quiz .
Database hosting	Neon	Managed PostgreSQL for production (Phase 14).
API hosting	Render	Deploys the Express API from GitHub (Phase 14).
Admin dashboard hosting	Vercel or Netlify	Static hosting for the React admin build (Phase 14).

Explicitly not used: microservices, GraphQL, Redis, Kubernetes, Firebase, Supabase, or other infrastructure beyond what a final-year-scale full-stack project needs.

4. Development Phases — Summary

Development proceeds one phase at a time, in the order below. Each phase is expanded in full in Section 5.

#	Phase	Purpose
1	Project Foundation & Environment Setup	Establish the technical foundation and repository structure so every later phase has a consistent, working base to build on.
2	Database Architecture & PostgreSQL Schema	Translate the spec's relational design into an enforced Prisma schema, migrations, and a development seed mechanism.
3	Backend API Foundation	Build the Express application shell — routing, middleware, validation, and error-handling conventions — that every endpoint in later phases will reuse.
4	Authentication & User Registration	Implement the client-required registration and login flow, including the explicit age restriction and alphanumeric password rule.
5	Mobile Application Foundation & Navigation	Stand up the Expo Router screen structure and navigation flow so registration/login and later quiz/home screens have somewhere to live.
6	Student Home & Session Management	Implement the authenticated home screen and the start-vs-continue decision point.
7	Quiz Engine — Question Delivery & Timing	Implement age-eligible question retrieval, quiz attempt creation, and a configurable, server-tracked timer.
8	Answering, Feedback & Progress Persistence	Implement answer submission, the client's exact required feedback wording, the wrong-answer retry behavior, and resumable progress.
9	Scoring & Results	Implement authoritative server-side grading, the quiz-completion flow, and the client's explicit above-70% cheers/claps trigger.
10	Audio, SVG & Visual Feedback Integration	Replace placeholder icons/sounds with the client-supplied custom assets, and add the animation polish appropriate for a 5–12 audience.
11	Admin Dashboard	Build the separate web admin application so the question bank and student/attempt data can be managed without editing code or touching the database directly.
12	Security Hardening & Validation	Run a dedicated pass across the whole system against the spec's explicit security checklist before calling anything done.
13	Integration & End-to-End Testing	Run the full test matrix against the completed system, across mobile, backend, and admin together.
14	Production Deployment & Handover	Get the system running somewhere the client can actually use it, and hand over a maintainable project.

5. Detailed Phase Descriptions

Each phase below follows the same structure: purpose, what is built, why it is needed, the technologies and components involved, the files/modules affected, how it connects to neighbouring phases, the functionality that should exist on completion, what to test, what to verify before moving on, and known risks or dependencies.

PHASE 1

Project Foundation & Environment Setup

PURPOSE

Establish the technical foundation and repository structure so every later phase has a consistent, working base to build on.

WHAT WILL BE BUILT

Monorepo with three workspaces (mobile/, server/, admin/); TypeScript configured in all three; Expo project initialized (React Native + Expo Router); Express server skeleton; local PostgreSQL instance; Prisma installed and connected; .env.example with placeholder variables; Git repository with .gitignore.

WHY IT IS NEEDED

Without a consistent foundation, later phases risk inconsistent tooling and rework. This also locks in the architectural constraint from the spec: no Firebase, no Supabase — a conventional backend API and PostgreSQL database from day one.

TECHNOLOGIES / COMPONENTS

Node.js, TypeScript, Expo / React Native, Expo Router, Express, PostgreSQL, Prisma, Git.

FILES / MODULES

root package.json (workspaces), mobile/app.json, mobile/tsconfig.json, server/src/index.ts, server/tsconfig.json, prisma/schema.prisma (initial), .env.example, README.md skeleton.

CONNECTION TO OTHER PHASES

Nothing precedes it. Every subsequent phase depends on it.

FUNCTIONALITY WHEN COMPLETE

The Express server and Expo dev server both start; an initial Prisma schema connects to a local PostgreSQL instance without error.

WHAT SHOULD BE TESTED

Server responds to a health-check route; Expo app loads a blank screen on simulator/device; prisma migrate dev succeeds against the local database.

VERIFY BEFORE MOVING ON

All three workspaces run without errors; database connection confirmed; environment variables documented in .env.example.

RISKS / DEPENDENCIES

Requires PostgreSQL available locally (or via Docker) and correct Node/Expo tooling versions. iOS simulator/Xcode setup is the most likely early blocker.

PHASE 2

Database Architecture & PostgreSQL Schema

PURPOSE

Translate the spec's relational design into an enforced Prisma schema, migrations, and a development seed mechanism.

WHAT WILL BE BUILT

Prisma models for users, questions, quiz_attempts, and answers with the relationships documented in the spec; the initial migration; prisma/seed.ts containing sample (clearly-marked-as-sample) Bible questions for development/testing only.

WHY IT IS NEEDED

Authentication, the quiz engine, and the admin dashboard all read and write through this schema. Getting relationships and constraints right early avoids painful migrations later.

TECHNOLOGIES / COMPONENTS

Prisma ORM, PostgreSQL.

FILES / MODULES

server/prisma/schema.prisma, server/prisma/migrations/, server/prisma/seed.ts.

CONNECTION TO OTHER PHASES

Builds on Phase 1's Prisma/Postgres connection. Every later backend phase (auth, quiz engine, admin) depends on these models.

FUNCTIONALITY WHEN COMPLETE

prisma migrate dev creates all four tables with correct types and relations; prisma db seed populates sample users and questions.

WHAT SHOULD BE TESTED

Inspect tables via pgAdmin/psql; verify foreign keys (quiz_attempts.user_id, answers.attempt_id, answers.question_id); confirm the schema cleanly supports keeping correct_option server-side only.

VERIFY BEFORE MOVING ON

Schema matches the spec's data model field-for-field, or deviations are explicitly flagged; seed script runs cleanly on a fresh database.

RISKS / DEPENDENCIES

Changing this schema after Phase 4+ is built on it means new migrations, not edits in place — worth getting age_min/age_max and the attempt status field right now.

PHASE 3

Backend API Foundation

PURPOSE

Build the Express application shell — routing, middleware, validation, and error-handling conventions — that every endpoint in later phases will reuse.

WHAT WILL BE BUILT

Express app entry point; route module structure (auth, me, quiz, admin); Zod-based request validation middleware; centralized error-handling middleware; a consistent JSON error/response shape; environment config loader.

WHY IT IS NEEDED

Establishes conventions once so later phases don't each reinvent validation and error handling — the spec explicitly calls for Zod validation and consistent error responses across the API.

TECHNOLOGIES / COMPONENTS

Express, TypeScript, Zod.

FILES / MODULES

server/src/index.ts, server/src/routes/*, server/src/middleware/errorHandler.ts, server/src/middleware/validate.ts, server/src/utils/*.

CONNECTION TO OTHER PHASES

Depends on Phase 1 (server skeleton) and Phase 2 (typed Prisma client). Phases 4, 6, 7, 8, 9, and 11 all build their endpoints on this foundation.

FUNCTIONALITY WHEN COMPLETE

The server exposes a consistent JSON error format; a sample validated route demonstrates the Zod + error-handling pattern end-to-end.

WHAT SHOULD BE TESTED

Postman requests against a health route and a deliberately invalid request, confirming the error shape; a unit test for the validation middleware.

VERIFY BEFORE MOVING ON

Validation and error-handling conventions are documented (even briefly) so later phases apply them consistently.

RISKS / DEPENDENCIES

Low risk structurally, but rushing it causes inconsistency across every phase that follows.

PHASE 4

Authentication & User Registration

PURPOSE

Implement the client-required registration and login flow, including the explicit age restriction and alphanumeric password rule.

WHAT WILL BE BUILT

POST /api/auth/register (first/middle/last name, age, mobile number, hobbies, favourite colour, favourite animal, password), POST /api/auth/login (first name + password, per the client's notes), POST /api/auth/logout, GET /api/auth/me; server-side age validation (5–12 inclusive, with the client's specified rejection message for ages above 12); secure password hashing; HTTP-only session/auth cookie.

WHY IT IS NEEDED

This is an explicit client requirement and gates every student-facing feature that follows.

TECHNOLOGIES / COMPONENTS

Express, Prisma, bcrypt/argon2, Zod, HTTP-only cookies (or an equivalent session strategy).

FILES / MODULES

server/src/routes/auth.ts, server/src/controllers/authController.ts, server/src/services/authService.ts, server/src/validators/authValidators.ts, server/src/middleware/requireAuth.ts.

CONNECTION TO OTHER PHASES

Built on Phase 3's conventions and Phase 2's users table. Every mobile screen from Phase 6 onward requires a logged-in user from this phase.

FUNCTIONALITY WHEN COMPLETE

A child aged 5–12 can register and receive a session; a child outside that range is rejected server-side even if client-side validation is bypassed; a registered user can log in with first name + password and log out.

WHAT SHOULD BE TESTED

Register with a valid age; register with age 3 and age 15 (both rejected); duplicate first names must not break login, since the internal ID — not the first name — is unique; wrong password; confirm passwords are stored only as hashes.

VERIFY BEFORE MOVING ON

No plaintext password appears anywhere in the database or logs; age validation cannot be bypassed by calling the API directly.

RISKS / DEPENDENCIES

First-name-only login is a real UX ambiguity if two children share a first name. This phase uses an internal unique ID and treats a collision as a flagged Open Requirement rather than silently forcing unique first names.

PHASE 5

Mobile Application Foundation & Navigation

PURPOSE

Stand up the Expo Router screen structure and navigation flow so registration/login and later quiz/home screens have somewhere to live.

WHAT WILL BE BUILT

Expo Router route structure matching the approved UI design's flow (Splash, Welcome, two-step Sign Up, Login, Home, Quiz, Result, plus loading/empty/network-error states); a shared layout/theme built from the supplied design tokens — cobalt #3a4ca0 (primary), lime #c6f24e (progress/confirmation), logo orange #f8981d (primary call-to-action), ink #14172e (borders) — rather than invented colors; an API client service (fetch wrapper with session handling).

WHY IT IS NEEDED

Without a navigation skeleton, screens built in later phases have nowhere to attach and no shared conventions for API calls or styling.

TECHNOLOGIES / COMPONENTS

React Native, Expo Router, TypeScript.

FILES / MODULES

mobile/app/_layout.tsx, mobile/app/(auth)/*, mobile/app/(app)/*, mobile/services/api.ts, mobile/constants/theme.ts, mobile/components/*.

CONNECTION TO OTHER PHASES

Depends on Phase 1. Feeds directly into Phase 6 (home), Phases 7–8 (quiz screens), and Phase 9 (results) — each is a screen dropped into this shell.

FUNCTIONALITY WHEN COMPLETE

The app launches to a Welcome screen; navigation between placeholder Sign Up / Login / Home screens works; the API client successfully calls the Phase 4 auth endpoints from a device or simulator.

WHAT SHOULD BE TESTED

Manual navigation walk-through on the iOS simulator; confirm the session persists across app restarts.

VERIFY BEFORE MOVING ON

Registration and login screens, wired to Phase 4, work end-to-end from the actual mobile app — not only from Postman.

RISKS / DEPENDENCIES

HTTP-only cookies behave differently in React Native than in a browser. This phase must confirm the chosen auth-persistence strategy actually works on-device early, since every later screen depends on it.

PHASE 6

Student Home & Session Management

PURPOSE

Implement the authenticated home screen and the start-vs-continue decision point.

WHAT WILL BE BUILT

GET /api/me and GET /api/me/attempts endpoints; a Home screen showing the student's first name and either "Start Quiz" or "Continue Quiz" depending on whether an in-progress attempt exists; auth persistence so a returning user goes straight to Home. The approved UI design also shows a "Stars Earned" count, a completed-quiz count, and a "Last Result" card on this screen — additional scope beyond the client's original minimal notes, included here because it appears in the client-approved design.

WHY IT IS NEEDED

This is the hub every quiz session starts or resumes from, and directly implements the client's explicit finished/continue status requirement.

TECHNOLOGIES / COMPONENTS

Express, Prisma, React Native.

FILES / MODULES

server/src/routes/me.ts, mobile/app/(app)/home.tsx, mobile/hooks/useAttemptStatus.ts.

CONNECTION TO OTHER PHASES

Depends on Phase 4 (auth) and Phase 5 (navigation shell). Directly precedes Phase 7, since Start/Continue Quiz is the entry point into the quiz engine.

FUNCTIONALITY WHEN COMPLETE

A logged-in student sees their first name and the correct button state, driven by real attempt data from the database.

WHAT SHOULD BE TESTED

A new user sees "Start Quiz"; a user with an unfinished attempt sees "Continue Quiz" and lands on the correct question when they tap it (fully verified once Phases 7–8 exist).

VERIFY BEFORE MOVING ON

Status is driven by actual quiz_attempts data, not a client-side guess.

RISKS / DEPENDENCIES

Low risk once authentication exists.

PHASE 7

Quiz Engine — Question Delivery & Timing

PURPOSE

Implement age-eligible question retrieval, quiz attempt creation, and a configurable, server-tracked timer.

WHAT WILL BE BUILT

POST /api/quiz/start (creates a quiz_attempts row and selects eligible questions where ageMin <= student.age <= ageMax, capped at the configured limit up to 70); GET /api/quiz/:attemptId (returns the current question, never including the correct answer); a server-recorded attempt start time; a mobile Quiz screen with a visible countdown.

WHY IT IS NEEDED

This is the core of the product and an explicit client requirement, and it is the phase most exposed to the spec's "do not trust the client" security requirements.

TECHNOLOGIES / COMPONENTS

Express, Prisma, React Native, a countdown/timer hook.

FILES / MODULES

server/src/routes/quiz.ts, server/src/controllers/quizController.ts, server/src/services/quizService.ts, mobile/app/(app)/quiz/[attemptId].tsx, mobile/components/Timer.tsx.

CONNECTION TO OTHER PHASES

Depends on Phase 4 (auth), Phase 6 (entry point), and Phase 2 (questions/quiz_attempts schema). Feeds directly into Phase 8 (answering).

FUNCTIONALITY WHEN COMPLETE

Starting a quiz creates a real attempt scoped to the student's age; questions are served without their correct answers; a countdown is visible and driven by a server-recorded start time, not only a client-side clock.

WHAT SHOULD BE TESTED

A student aged 6 never receives a question outside its age range; requesting more than 70 questions is capped; the network response confirms the correct answer is absent; timer expiry is handled gracefully.

VERIFY BEFORE MOVING ON

The correct answer never appears in any question payload sent to the mobile client, under any code path.

RISKS / DEPENDENCIES

The exact timer duration and the behavior on timer expiry are not defined by the client notes — flagged as an Open Requirement. The approved UI design mocks up 30 seconds per question and a 20-question quiz as illustrative example values, which is a reasonable configurable default but not a confirmed final requirement. Implemented with a configurable value and a clearly stubbed expiry behavior rather than a guessed final UX.

PHASE 8

Answering, Feedback & Progress Persistence

PURPOSE

Implement answer submission, the client's exact required feedback wording, the wrong-answer retry behavior, and resumable progress.

WHAT WILL BE BUILT

POST /api/quiz/:attemptId/answer, validating the submitted option server-side against the database and never trusting client-side correctness; the "Well done" + star feedback on correct answers; the "Whoops! That is the wrong answer, let's try again." feedback on incorrect answers, staying on the current question; persistence of the current question index, selected answers, and last-activity time so an exited quiz resumes exactly where it left off.

WHY IT IS NEEDED

These are explicit, verbatim client requirements for wording and behavior, and the persistence requirement is what makes "Continue Quiz" from Phase 6 actually work.

TECHNOLOGIES / COMPONENTS

Express, Prisma, React Native.

FILES / MODULES

server/src/controllers/quizController.ts (extended), mobile/app/(app)/quiz/[attemptId].tsx (extended), mobile/components/AnswerFeedback.tsx.

CONNECTION TO OTHER PHASES

Extends Phase 7 directly. Feeds into Phase 9 — completion happens once every question is answered here.

FUNCTIONALITY WHEN COMPLETE

Every answer submission is graded server-side; correct/incorrect feedback matches the client's exact wording; closing and reopening the app mid-quiz resumes at the correct question with prior answers intact.

WHAT SHOULD BE TESTED

Submit a manipulated option directly via Postman to confirm the server, not the app, determines correctness; force-quit the app mid-quiz and confirm resume; verify a wrong answer does not advance the question index.

VERIFY BEFORE MOVING ON

A student cannot influence their score by calling the API directly with a fabricated "correct" flag.

RISKS / DEPENDENCIES

The exact wrong-answer retry limit (unlimited vs. capped attempts) is not specified by the client. Implemented as unlimited retries, staying on the question, per the spec's provisional interpretation — flagged as an Open Requirement.

PHASE 9

Scoring & Results

PURPOSE

Implement authoritative server-side grading, the quiz-completion flow, and the client's explicit above-70% cheers/claps trigger.

WHAT WILL BE BUILT

POST /api/quiz/:attemptId/complete (marks the attempt finished and computes correct/total/percentage server-side); GET /api/quiz/:attemptId/result; a Result screen showing score, percentage, and completion state, matching the two result variants in the approved UI design — an above-70% state (cheers audio) and a below-70% state showing "Good try, [First Name]. Play again for more stars." with no audio; client-side audio playback gated strictly on the server-returned percentage exceeding 70.

WHY IT IS NEEDED

Explicit client requirements — the score must be trustworthy, and the cheers/claps behavior is one of the most specific requirements in the client's notes.

TECHNOLOGIES / COMPONENTS

Express, Prisma, React Native, an audio-playback library (e.g. expo-av).

FILES / MODULES

server/src/controllers/quizController.ts (extended), mobile/app/(app)/quiz/result/[attemptId].tsx, mobile/services/audio.ts.

CONNECTION TO OTHER PHASES

Depends on Phases 7–8 (attempt/answer data). This is the natural point to wire in the cheers/claps audio asset once supplied, tying into Phase 10.

FUNCTIONALITY WHEN COMPLETE

Completing a quiz produces a stored, server-calculated result; a score of 75% plays the cheers sound, while exactly 70% does not, per the literal "above 70" wording in the notes.

WHAT SHOULD BE TESTED

Attempts scoring 65%, 70%, 70.1%, and 100%, confirming the audio only fires above the threshold; confirm the percentage is computed from stored answers, never a client-submitted number.

VERIFY BEFORE MOVING ON

The 70% boundary behaves exactly as specified — this is one of the most literally-quoted requirements in the client's notes.

RISKS / DEPENDENCIES

Depends on the Cheers/Claps Audio asset (Required). Can be built and tested with a placeholder sound first, then swapped once supplied.

PHASE 10

Audio, SVG & Visual Feedback Integration

PURPOSE

Replace placeholder icons/sounds with the client-supplied custom assets, and add the animation polish appropriate for a 5–12 audience.

WHAT WILL BE BUILT

The Correct Answer Star SVG wired into the correct-answer feedback from Phase 8; the Age Error Smiley SVG wired into the age-rejection screen from Phase 4; the Cheers/Claps audio wired into the result screen from Phase 9; the mascot illustration (welcome screen, home card, empty state — 3 poses per the approved UI design) and the logo, wired into their placeholders; light, child-appropriate animations — not the excessive animation the spec explicitly cautions against.

WHY IT IS NEEDED

The client explicitly asked that these custom assets not be replaced with generic icons or emoji, and that development wait for the developer to supply them before finalizing this UI.

TECHNOLOGIES / COMPONENTS

react-native-svg (or equivalent), expo-av for audio, Reanimated or the Animated API for lightweight transitions.

FILES / MODULES

mobile/assets/*, mobile/components/AnswerFeedback.tsx (updated), mobile/components/AgeErrorMessage.tsx (updated), mobile/services/audio.ts (updated).

CONNECTION TO OTHER PHASES

Cannot be finalized until the Required assets from the Asset Request List are supplied. Depends on Phases 4, 8, and 9 already having placeholder feedback in place to swap into.

FUNCTIONALITY WHEN COMPLETE

All explicitly-required client assets are visible/audible in their correct locations, replacing any placeholder used during earlier phases.

WHAT SHOULD BE TESTED

On-device visual check that SVGs render crisply at the suggested sizes and with correct colors (see risk below); audio check on a real device speaker, not only a desktop; confirm animations don't delay the Next Question flow.

VERIFY BEFORE MOVING ON

No placeholder emoji remain in the correct-answer, age-error, or cheers paths.

RISKS / DEPENDENCIES

Directly blocked on asset delivery — intentionally sequenced late so it never blocks the functional phases before it. Technical flag confirmed by inspecting the supplied files: star.svg, logo.svg, and emoji.svg all define their fill colors through an internal `<style>` block with CSS classes (e.g. `.cls-1{fill:#f39a1f}`) rather than inline attributes. react-native-svg has historically had inconsistent support for `<style>`-based class fills, and the same issue was independently flagged by the UI design tool when importing emoji.svg. Before wiring these into the app, re-export each SVG with colors as inline presentation attributes (in Illustrator: Styling ! Presentation Attributes on export) and confirm they render correctly in an actual react-native-svg test, not just a browser preview. The UI design also calls for an App Icon (1024×1024 PNG, no transparency, logo mark only — no wordmark), which is a distinct export from the existing logo files and should be requested if not already supplied.

PHASE 11

Admin Dashboard

PURPOSE

Build the separate web admin application so the question bank and student/attempt data can be managed without editing code or touching the database directly.

WHAT WILL BE BUILT

Admin login, separate from student login; an Overview tab (completion rate, attempts/completed/questions/students counts, and a by-age-band breakdown chart — the chart's age bands are a display-only grouping for the analytics view, not a change to the underlying freeform ageMin/ageMax data model); a Questions tab (list with age range and correct answer per card, Edit/Deactivate, and an Add/Edit form with A–D options, a correct-answer selector, Min/Max Age fields, and an Active toggle); a Students tab (read-only, explicitly never showing password hashes or contact details, per the approved UI design's own privacy note); an Attempts tab (filterable list showing student, age, progress, and duration).

WHY IT IS NEEDED

Explicit client requirement — populating the database will be done by the client — the admin dashboard is how that happens without direct SQL access.

TECHNOLOGIES / COMPONENTS

React, TypeScript, a lightweight UI library, the same backend API with admin-scoped endpoints.

FILES / MODULES

server/src/routes/admin.ts, server/src/middleware/requireAdmin.ts, admin/src/pages/*, admin/src/components/*, admin/src/services/api.ts.

CONNECTION TO OTHER PHASES

Depends on Phase 2 (schema), Phase 3 (API conventions), and Phase 4's auth pattern, extended with an admin role check. Once complete, this becomes the client's primary way to load real Bible questions, superseding the Phase 2 seed data.

FUNCTIONALITY WHEN COMPLETE

An admin can log in, add/edit/deactivate a question with an age range, and see registered students and quiz results, without touching Prisma Studio or SQL directly.

WHAT SHOULD BE TESTED

A non-admin user hitting /api/admin/* routes is rejected; question CRUD round-trips to the same table the mobile app reads from; deactivated questions stop appearing in student quizzes.

VERIFY BEFORE MOVING ON

Admin routes are provably inaccessible to a regular student session, not only hidden in the UI.

RISKS / DEPENDENCIES

Requires a small schema decision — an admin role/flag on users, or a separate admins table — confirmed before building this phase.

PHASE 12

Security Hardening & Validation

PURPOSE

Run a dedicated pass across the whole system against the spec's explicit security checklist before calling anything done.

WHAT WILL BE BUILT

Confirmation, and fixes where needed, that: passwords are hashed everywhere; client- and server-side validation both exist; server-side age validation is unbyypassable; the correct answer is never exposed; client-submitted scores are never trusted; ownership checks prevent a student from accessing another student's attempt; admin routes are protected; .env secrets stay out of version control; inputs are sanitized; HTTPS is used in production.

WHY IT IS NEEDED

Named as its own priority in the spec — security is not an afterthought bolted onto polish.

TECHNOLOGIES / COMPONENTS

Manual review plus targeted tests (Postman scripts hitting endpoints with another user's IDs, missing/invalid tokens, tampered payloads).

FILES / MODULES

No new features — targeted fixes across server/src/middleware/*, server/src/controllers/*, .gitignore, .env.example.

CONNECTION TO OTHER PHASES

Cuts across every backend phase (3–4, 7–9, 11). Must happen before Phase 14.

FUNCTIONALITY WHEN COMPLETE

A written checklist, mirroring the spec's security section, with each item confirmed against the running system, not assumed from the code.

WHAT SHOULD BE TESTED

Attempt to fetch another student's quiz attempt by guessing an ID (must be rejected); attempt to POST a fabricated 100% score directly (must be ignored); confirm .env is git-ignored and .env.example has no real secrets.

VERIFY BEFORE MOVING ON

Every security item has been actively tested, not just "should work by construction."

RISKS / DEPENDENCIES

None architecturally — a review/hardening phase that should only reduce risk, not introduce it.

PHASE 13

Integration & End-to-End Testing

PURPOSE

Run the full test matrix against the completed system, across mobile, backend, and admin together.

WHAT WILL BE BUILT

End-to-end walk-throughs covering registration, invalid age, duplicate/edge-case users, login/logout, wrong password, starting a quiz, timer behavior, correct/wrong answers, retry behavior, closing/reopening the app, continue behavior, quiz completion, scores, the above-70% audio, admin question creation/editing/deletion, unauthorized admin access, and basic database/network failure handling.

WHY IT IS NEEDED

Individual phases were verified in isolation; this phase confirms the pieces work together as one product — explicitly listed in the client's own phase list.

TECHNOLOGIES / COMPONENTS

Manual QA on a real device/simulator plus a Postman regression collection covering all endpoints.

FILES / MODULES

A test log/checklist document (e.g. TESTING.md), plus bug-fix commits as issues are found.

CONNECTION TO OTHER PHASES

Exercises every phase from 1–12 together. Must pass before Phase 14.

FUNCTIONALITY WHEN COMPLETE

A documented pass/fail record against the full scenario list, with any failures fixed and re-verified.

WHAT SHOULD BE TESTED

This phase is the testing — see the scenario list above.

VERIFY BEFORE MOVING ON

No scenario in the list is left unverified or marked "assumed working."

RISKS / DEPENDENCIES

Time-boxed but not skippable — integration bugs between mobile, server, and admin surface here, and are cheaper to find here than after deployment.

PHASE 14

Production Deployment & Handover

PURPOSE

Get the system running somewhere the client can actually use it, and hand over a maintainable project.

WHAT WILL BE BUILT

A production PostgreSQL database on Neon; the Express API deployed to Render (HTTPS, environment variables set, nothing hard-coded), upgraded off Render's free tier before real handover so it doesn't cold-start on the client's first login; the admin dashboard deployed as a static build to Vercel or Netlify; an Expo build (development/preview build at minimum, App Store submission only if the client wants to go that far); a README covering setup, environment variables, architecture, and how to run migrations/seed; a short handover walk-through of the codebase.

WHY IT IS NEEDED

Explicit Definition of Done items — a clear README, PostgreSQL remaining directly manageable, and the requirement that the project owner can keep managing the system after development ends.

TECHNOLOGIES / COMPONENTS

Neon (PostgreSQL), Render (Express API), Vercel or Netlify (admin dashboard static build), Expo build tooling (EAS Build).

FILES / MODULES

README.md (completed), render.yaml or Render dashboard config, .env.example (finalized).

CONNECTION TO OTHER PHASES

Final phase — depends on everything before it being complete and tested in Phase 13.

FUNCTIONALITY WHEN COMPLETE

The system is reachable outside localhost; the client can log into the admin dashboard and manage real questions; PostgreSQL remains accessible via normal admin tools (pgAdmin, via the Neon connection string), per the spec's explicit requirement.

WHAT SHOULD BE TESTED

A fresh-machine setup test — can a new developer get the project running by following only the README? A production smoke test of registration, quiz, result, and an admin question edit, run against the Render/Neon deployment rather than localhost.

VERIFY BEFORE MOVING ON

The full Definition of Done, used as the final gate before calling the project complete.

RISKS / DEPENDENCIES

Hosting is now decided (Neon + Render + Vercel/Netlify), removing the main open risk from this phase. The remaining watch-item is Render's free-tier cold start — confirm the Starter tier is active before the client's actual first login.

6. Dependencies Between Phases

Development is largely linear. The table below makes each phase's prerequisites explicit so work is not started out of order.

#	Phase	Depends on
1	Project Foundation	None — starting point
2	Database Architecture	Phase 1
3	Backend API Foundation	Phases 1–2
4	Authentication	Phases 2–3
5	Mobile Foundation & Navigation	Phase 1
6	Student Home	Phases 4–5
7	Quiz Engine — Delivery & Timing	Phases 2, 4, 6
8	Answering & Progress Persistence	Phase 7
9	Scoring & Results	Phases 7–8
10	Audio / SVG / Visual Integration	Phases 4, 8, 9 + Required assets
11	Admin Dashboard	Phases 2–4
12	Security Hardening	Phases 3–4, 7–9, 11
13	Integration & E2E Testing	Phases 1–12
14	Deployment & Handover	Phase 13

7. Testing Checkpoints

These scenarios, drawn directly from the client's build specification, form the checklist exercised in Phase 13 and re-checked before deployment in Phase 14.

- ☐ Registration succeeds for a valid age (5–12 inclusive).
- ☐ Registration is rejected for age below 5 and above 12, showing the specified message.
- ☐ Duplicate first names do not break login or registration.
- ☐ Login and logout work; a wrong password is rejected.
- ☐ A quiz can be started and eligible questions respect the student's age range.
- ☐ The question timer is visible and its expiry is handled gracefully.
- ☐ Correct answers show “Well done” with the star; incorrect answers show the specified retry message and stay on the same question.
- ☐ Closing and reopening the app resumes an in-progress quiz at the correct position.
- ☐ Quiz completion produces a server-calculated score, percentage, and status.
- ☐ A score above 70% plays the cheers/claps audio; a score of exactly 70% does not.
- ☐ An administrator can create, edit, and deactivate/delete questions.
- ☐ A non-admin session cannot reach admin-only endpoints.
- ☐ Database failures and network failures are handled without crashing the app.

8. Final Definition of Done

The project is functionally complete when every item below is true of the running system, not merely present in the code.

- ☐ A valid child aged 5–12 can register; an invalid age is rejected appropriately.
- ☐ Passwords are securely stored as hashes, never in plaintext.
- ☐ A registered student can log in with first name and password.
- ☐ The home screen displays the student's first name and the correct start/continue state.
- ☐ Questions are loaded from PostgreSQL through the backend API, respecting the configured age range and never exceeding 70 per quiz.
- ☐ The quiz has a configurable time limit with a visible countdown.
- ☐ Answers are validated by the backend; the client cannot set its own score.
- ☐ Correct and incorrect answers produce the client's exact required feedback.
- ☐ Quiz progress persists and an unfinished quiz can be continued; completed quizzes are marked finished.
- ☐ A score above 70% triggers the cheers/claps sound; results are stored.
- ☐ An administrator can populate and manage questions through the admin dashboard.
- ☐ PostgreSQL remains directly manageable through standard database administration tools.
- ☐ Authentication and authorization are enforced throughout, including on admin routes.
- ☐ The mobile app handles loading, error, and network-failure states.
- ☐ The project has a clear README and setup instructions.
- ☐ The architecture and major implementation decisions can be explained by the developer.

9. Deployment & Handover Considerations

Deployment is addressed in Phase 14, once Phase 13's full test pass is complete. The considerations below should be confirmed before the project is treated as handed over.

- Hosting is decided: PostgreSQL on Neon, the Express API on Render, and the admin dashboard on a static host (Vercel or Netlify — pick one when Phase 14 starts, either works with no code changes needed).
- Render's free tier sleeps the API after 15 minutes of inactivity (a ~30–50 second cold start on the next request). Upgrade to Render's paid Starter tier before the client's actual handover so her first login doesn't look broken.
- Neon's connection string can be handed to pgAdmin directly for direct database access, satisfying the client's requirement that the database stay reachable through standard admin tools — no VPN or server access needed.
- Environment variables (DATABASE_URL, SESSION_SECRET, API_URL, and similar) must be set on Render/Neon/Vercel directly, never hard-coded or committed.
- HTTPS must be enforced in production for all API traffic — Render and Neon provide this by default.
- A README covering setup, environment variables, architecture, and how to run migrations/seed data must accompany the handover.
- A short walk-through of the codebase with the project owner/developer is recommended so the system can be maintained independently afterward.

Appendix A — Open Requirements

These items are intentionally left unresolved in the current implementation because the client's notes do not specify a final answer. Each is implemented with a safe, reversible provisional behavior and should be confirmed with the client rather than treated as settled.

Open item	Current provisional behavior
Exact question/quiz timer duration	Configurable value stored server-side. The approved UI design mocks 30 seconds per question as an example — a reasonable default, not yet a confirmed final value.
Behavior when the timer expires	Stubbed pending client input — needs a decision (auto-submit blank, mark wrong, or extend).
Wrong-answer retry limit	Unlimited retries, staying on the same question, per the spec's provisional reading of the client's notes.
Questions per quiz (display default)	The UI design mocks a 20-question quiz as its example; the spec's cap of 70 remains the hard ceiling, with the actual per-quiz count still configurable by the admin.
Age bands for question difficulty	Still none in the data model — each question keeps a freeform ageMin/ageMax. The admin dashboard's Overview chart groups results into 5–7 / 8–10 / 11–12 bands for display only; this is a reporting convenience, not a change to how questions are stored or filtered.
First-name login collisions	Internal unique user ID used behind the scenes; first-name-only login kept as the visible UX unless the client requests a change.
SVG asset color rendering in React Native	star.svg, logo.svg, and emoji.svg all define colors via an internal <style> block with CSS classes rather than inline attributes — a pattern react-native-svg has inconsistent support for. Re-export each with colors as inline presentation attributes before Phase 10 and verify on-device, not just in a browser preview.
Production hosting platform	Resolved: PostgreSQL on Neon, Express API on Render, admin dashboard on Vercel or Netlify (final pick deferred to Phase 14, either works without code changes).
Default admin account credentials	A single shared admin login is seeded (not per-person accounts), with a placeholder password meant to be changed on first real use via the admin panel's own settings screen. The password is hashed like any other; the placeholder value itself is never committed to the Git repository.

Appendix B — Asset Request List

The custom visual and audio assets requested from the developer. The first three rows were requested directly from the client's spec; the remaining rows were identified once the UI design (Appendix C) was supplied and are recommended rather than strictly required. Status reflects this document's most recent update — see the cover page for the version.

Asset	Status	Format	Usage
Correct Answer Star	Received — assets/star.svg (re-export needed, see note)	SVG	Correct-answer “Well done” feedback (Phase 10)
Age Error Smiley	Received — assets/emoji.svg (re-export needed, see note)	SVG	Age > 12 registration error message (Phase 10)
Cheers / Claps Audio	Received & trimmed — assets/cheers.mp3, 4.0s	MP3	Final score above 70% (Phase 10)
App Logo / Branding	Received (bonus) — logo.png / logo.svg (re-export needed) / logo.ai	SVG / PNG	Welcome screen, splash
Mascot (3 poses)	Recommended — not yet supplied	SVG, 1:1, ~116px	Welcome screen, home card, empty state (per UI design)
App Icon	Recommended — not yet supplied	PNG, 10 24×1024, opaque	iOS app icon — logo mark only, no wordmark, no transparency
Empty-state illustration	Optional — not supplied	SVG, 1:1	“No questions for this age” state
Custom Navigation Icons	Optional — not supplied	SVG, 24px grid	Back, show/hide password, close, star-count — otherwise a standard icon library is used

NOTE ON THE CHEERS / CLAPS AUDIO ASSET

Resolved: the original supplied clip ran 25.7 seconds, well past the 2–4 second sting suggested in the original request. It has since been manually trimmed to assets/cheers.mp3 (4.0 seconds, 192kbps), which is ready to wire in during Phase 10 as-is.

NOTE ON SVG COLOR RENDERING (STAR.SVG, LOGO.SVG, EMOJI.SVG)

All three supplied SVGs define their colors through an internal `<style>` block with CSS classes (e.g. `.cls-2{fill:#ffc713}`) rather than inline attributes on each path. This was independently flagged by the UI design tool when importing emoji.svg (“colours live in an internal stylesheet that gets stripped on import”), and the same pattern is present in star.svg and logo.svg. react-native-svg — the library this project's spec calls for — has a history of incomplete support for `<style>`/class-based fills. Before Phase 10 wires these in, re-export each file with colors as inline presentation attributes (in Illustrator: select all, then Object !' ungroup styles, or use File !' Export !' Styling: Presentation Attributes) and confirm the colors render correctly in an actual React Native build,

not only a browser preview.

The three client-required assets are received; the audio is trimmed and ready. Mascot and App Icon are recommended follow-ups surfaced by the UI design rather than the original client notes, and are not blockers for any phase before Phase 10. This document is a living plan; as phases are completed, assets are supplied, or requirements are clarified with the client, this PDF should be regenerated and re-shared so it stays the single source of truth for the project's current state.

Appendix C — UI Design Reference

The client supplied a full UI design covering the mobile app and admin dashboard (Claude Design canvas, “Cobalt Playground” direction, chosen after comparing three color/style directions). This appendix summarizes what it specifies, so implementation phases can build against it directly rather than inventing layout or copy.

DESIGN TOKENS

Token	Value	Usage
Cobalt (primary)	#3a4ca0	Headers, primary backgrounds, borders on key cards
Lime	#c6f24e	Progress bars, in-progress badges, correct-answer feedback background
Logo orange	#f8981d	Primary call-to-action buttons (Sign Up, Start Quiz, Save Question)
Ink	#14172e	Border/outline color used consistently across every screen

MOBILE SCREEN INVENTORY

- Splash — logo on a white card over a cobalt background.
- Welcome — mascot placeholder, tagline, Sign Up (orange) / Log In (white) buttons.
- Sign Up — two steps: Step 1 collects first/middle/last name, age, and mobile number (with a “a grown-up can help” note); Step 2 collects favourite colour, favourite animal, hobbies, and password/confirm-password with live match validation.
- Log In — first name + password, matching the client's exact requirement.
- Age Too High — smiley placeholder, “Oops, the age is too high” heading, “This quiz is for children aged 5 to 12. Ask a grown-up if you need help.” body, “Change My Age” button.
- Home (start) — greeting, a “20 questions · picked for age 9 · 30 seconds each” summary card, Start Quiz button, plus Stars Earned / Quizzes Completed stat cards and a Last Result card.
- Home (continue) — “Welcome back” greeting, a SAVED in-progress badge with question position, Continue Quiz (lime) button, and a secondary “Start a New Quiz” outlined button.
- Quiz Question — question counter, question text, four tappable options, “Tap the answer you think is right” helper text.
- Correct feedback (“Well Done”) — lime full-screen state with the star asset and a Next Question button.
- Wrong feedback (“Whoops”) — shown inline on the question screen as an orange banner reading the client's exact wording, staying on the same question.
- Quiz Complete — above 70% — percentage, “X of Y correct”, cheers audio.
- Quiz Complete — 70% or below — same layout, no audio, “Good try, [First Name]. Play again for more stars.”
- System states — loading skeleton, empty state (“No questions for this age”), and a network-error state (“We lost the internet — your quiz is saved” with a Try Again button) — covering the spec's loading/error/network-failure requirement.

ADMIN DASHBOARD INVENTORY

- Overview tab — completion rate, attempts/completed/questions/students counts, and a by-age-band chart (display grouping only — see Appendix A).

- Questions tab — card list (ID, age range, question text, correct answer, Edit/Deactivate) and an Add/Edit form (options A–D, correct-answer selector, Min/Max Age, an Active toggle captioned “shown in new quizzes”).
- Students tab — name, age, attempt count, last score; explicitly annotated in the design itself as showing no contact details or password data.
- Attempts tab — filterable list showing student, age, progress, and duration for each attempt.

This inventory should be treated as the target UI for Phases 5–11. Where it adds scope beyond the client's original handwritten notes (e.g. home-screen stats, the two-step sign-up split, system-state screens), that addition is noted in the relevant phase in Section 5 rather than silently assumed.

Appendix D — Quiz History (Student-Side Scope Addition)

This appendix documents a scope addition confirmed with the developer while testing Phases 6 through 8, following the same convention the rest of this plan uses for anything added beyond the client's original handwritten notes (see Section 5's phase-by-phase notes and Appendix A).

STATUS

Developer decision / scope addition. Requested by the developer during on-device testing of Phases 6 and 8, after noticing the Home screen's single "Last Result" card gave no way to look back at earlier attempts (2026-09-17). Not part of the client's original handwritten notes; not a blocker for Phase 9.

PURPOSE

Let a student look back at more than just their single most recent quiz result. The Phase 6 Home screen's Last Result card only ever shows the latest attempt — there was no way to see anything earlier from the mobile app itself.

WHAT WILL BE ADDED

A dedicated Quiz History screen listing every one of the student's past attempts (date, score, percentage, and status — finished vs. abandoned via "Start a New Quiz"), most recent first, reachable from a "View All Results" link on the Home screen.

WHY IT IS NEEDED

The client asked for this on the student side specifically — not only through the admin dashboard's Attempts tab (Phase 11), which the student never sees.

TECHNOLOGIES / COMPONENTS

React Native (mobile only). No new backend endpoint is required: GET /api/me/attempts (Phase 6) already returns the student's complete attempt history — this is a presentation-layer addition on top of existing, already-secured data.

FILES / MODULES

mobile/src/app/(app)/history.tsx (new); a reusable past-attempt list-item card under mobile/src/components/ui/; a small addition to the Home screen linking to it.

CONNECTION TO OTHER PHASES

Extends Phase 6 (Student Home & Session Management). Depends on the Phase 8 data-integrity fix (2026-09-17) that finalizes an attempt's status, score, and percentage the moment its last question is answered — before that fix, a completed quiz could stay stuck IN_PROGRESS and would have shown up incorrectly in a history list too.

WHAT SHOULD BE TESTED

Empty state for a brand-new student with no attempts yet; correct ordering (most recent first); finished vs. abandoned attempts are visually distinguishable; tapping a past entry does not let the student re-answer or otherwise alter it.

RISKS / DEPENDENCIES

None architecturally — this reuses existing, already-scoped data (GET /api/me/attempts already restricts results to the authenticated student only, per spec Section 20). Best sequenced alongside or after Phase 9, once a finished attempt's percentage is something worth looking back on.

Appendix E — Physical Testing Checklist

A running checklist of items that need a human, a real browser, or a real device to confirm — things automated testing (type-checks, direct API requests, or a scripted browser session) cannot fully exercise on its own. One section is added per completed phase; each item should be checked once, either right after the phase that introduced it or during Phase 13's full Integration & End-to-End Testing pass, alongside the Section 7 checklist.

Phase 11 — Admin Dashboard

Phase 11 was verified end-to-end in an automated browser session — login, all four tabs, question CRUD, search/filters, and the full password-change-then-relogin cycle all confirmed working directly against the running API. A handful of items need a real browser, a human, or the actual mobile app to confirm, since a scripted session cannot exercise them.

- ☐ **Delete confirmation dialog.**
The automated testing session runs in a browser that auto-dismisses native JavaScript dialogs, so Questions > Delete was exercised but its "OK" path was never actually clicked. Confirm the browser's native confirm() box appears with the question text, Cancel leaves the question untouched, and OK deletes it.
- ☐ **Change the seeded admin's placeholder password.**
Log in with the placeholder credentials, go to Settings, set a real password, then confirm the placeholder no longer works and the new password does.
- ☐ **Question CRUD reaches the real mobile app.**
Add a question via the admin dashboard and confirm a new quiz on the actual phone offers it (within its age range); deactivate a question and confirm a new quiz stops offering it. This is the one Phase 11 check the development plan calls out explicitly ("question CRUD round-trips to the same table the mobile app reads from"), and it requires both apps running for real.
- ☐ **Dashboard at your own screen size.**
Automated testing only checked a fixed 1024×768 viewport — confirm the layout still reads well at your actual monitor resolution.
- ☐ **Cross-device access (informational, not a defect).**
CORS_ORIGIN currently only allows localhost:5173, so opening the dashboard from a phone or another computer on the network will fail until Phase 14's deployment widens it. Expected for now — nothing to fix.

RISKS / DEPENDENCIES

None architecturally — these are verification gaps left by automated testing (a scripted browser session suppresses native dialogs, and there is no way to drive the physical mobile app from that same session), not known defects.

Phase 12 — Security Hardening & Validation

Every item in the spec's security checklist (Section 20) was actively tested against the running server — not assumed from the code — with direct API requests: password hashing, both client- and server-side registration/age validation, correct-answer non-exposure, rejection of fabricated correctness/score fields, cross-student ownership checks, admin-route authorization (unauthenticated, invalid token, and wrong-role cases), .env hygiene, and input sanitization against malformed JSON, invalid enum values, and non-numeric IDs. One real (if dormant) issue was found and fixed: the CORS fallback allowed any origin with credentials when CORS_ORIGIN was unset — now it falls back to the same explicit dev origins .env.example documents instead. HTTPS in production is deferred to Phase 14 (Render and Neon both provide it by default; not testable against localhost).

No physical testing required for this phase.

RISKS / DEPENDENCIES

None — every check in this phase is verifiable through direct API requests, all of which were run and confirmed during implementation. No human, real browser, or physical device is needed here.

Phase 13 — Integration & End-to-End Testing

The full scenario matrix (see TESTING.md at the repo root) was run against the live API — registration and its age/duplicate-name/wrong-password edge cases, the complete quiz engine flow including the wrong-answer retry, a real 9-question attempt played to 100%, the exact >70% cheers boundary tested at 65/70/75%, close-and-reopen persistence, and admin CRUD with unauthorized-access rejection. What a scripted API session cannot confirm is whether those same guarantees hold when a person is actually looking at the screen and holding the phone.

- ☐ **Full student walkthrough on a real device.**
Register, land on Home, start a quiz, answer a question wrong then correctly, and reach the result screen. The API responses for each step were verified directly, but rendering, navigation, and wording on the actual screen were not.
- ☐ **Timer expiry UX on-device.**
Let a question's countdown reach 0 and confirm the "TIME'S UP" screen appears with only "Start a New Quiz" offered — not "Continue Quiz" for an attempt whose current question can no longer be answered. (Fixed and verified in an earlier session; re-check only if the timer code has changed since.)
- ☐ **Cheers/claps audio is actually audible.**
Complete a real quiz above 70% and confirm the cheers/claps sound plays through the device speaker; complete one at or below 70% and confirm it stays silent. The >70% boundary logic itself was tested precisely (65/70/75% synthetic attempts), but audio playback can only be confirmed on a real device.
- ☐ **Close and reopen the physical app mid-quiz.**
Force-quit or background the app partway through a quiz, then reopen it and tap Continue Quiz. The server-side persistence was confirmed via direct API calls simulating this, but the actual app

needs to be closed and reopened by hand to confirm the UI itself resumes correctly.

- ☐ **Network loss mid-quiz.**
Turn off Wi-Fi on the phone while a quiz is in progress and confirm the "We lost the internet — your quiz is saved" state appears, then confirm the quiz resumes cleanly once connectivity returns. Cannot be simulated from this session.

RISKS / DEPENDENCIES

None architecturally — these are the physical-device counterparts to scenarios already confirmed at the API level; nothing here is expected to fail unless a mobile-side regression has been introduced separately.